

Hands-on Lab - Creating a Swagger documentation for REST API using Python



Estimated Time: 45 minutes

In this lab, you will understand how to create a Swagger documentation for your REST APIs.

Learning Objectives:

After completing this exercise, you should be able to perform the following tasks:

- Use the Swagger Editor to create Swagger documentation for REST API
- Use SwaggerUI to access the REST API endpoints of an application
- Generate code with the Swagger documentation

Pre-requisites

- You must be familiar with Docker applications and commands
- You must have a good understanding of REST API.
- Knowledge of Python is highly recommended

Task 1 - Getting your application started

1. Open a terminal window by using the top menu in the IDE: **Terminal > New Terminal**, if you don't have one open already.

2. In the terminal, clone the repository which has the Swagger documentation and the REST API code ready by pasting the following command. The repository that you clone has code that will run a REST API application which can be used to organize tasks.

```
git clone https://github.com/ibm-developer-skills-network/jmgdo-microservices.git
```

3. Change the working directory to **jmgdo-microservices/swagger_example** by running the following command.

```
cd jmgdo-microservices/swagger_example
```

4. Run the following commands to install the required packages.

```
python3 -m pip install flask_cors
```

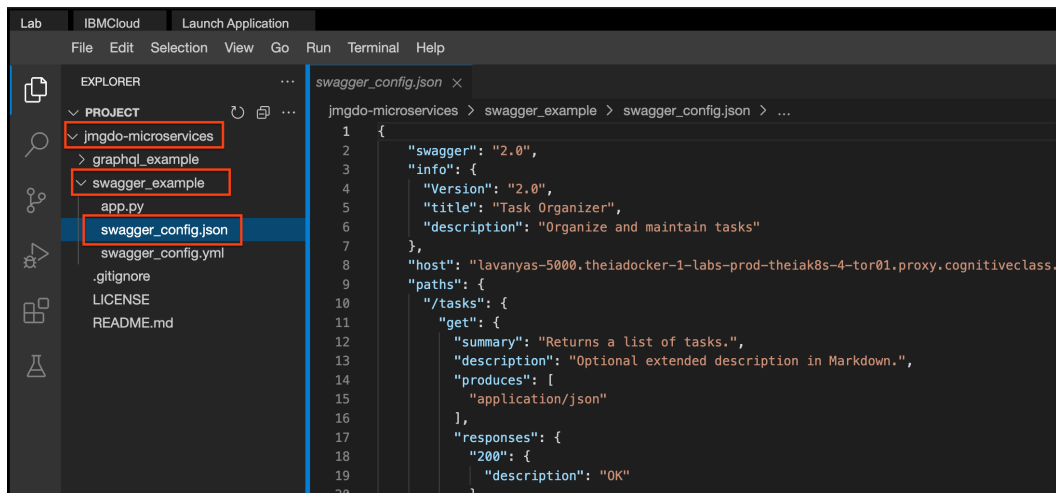
5. Now start the application which serves the REST API on port number 5000.

```
python3 app.py
```

6. Click on the Skills Network button on the left, it will open the "Skills Network Toolbox". Then click Launch Application, from there you enter the port no. as 5000 and click Your Application button. This will open a new browser page, which accesses the application you just ran.

7. Copy the url on the address bar.

8. From the file menu, go to **jmgdo-microservices/swagger_example/swagger_config.json** to view the file on the file editor.



9. In the file editor, paste the application URL that you copied where it says **your application URL** without the protocol (**https://**) and do not put a **7** at the end of the URL, and save the file.

10. Copy the entire content of the file **swagger_config.json**. You will need this copied content to generate SwaggerUI.

11. Click on this link <https://editor.swagger.io/> to go to the Swagger Editor.

12. From the File menu, click on Clear Editor to clear the content of the Swagger Editor.

13. Paste the content you copied from **swagger_config.json** on the left side. You will get a prompt which says **would you like to convert your JSON into YAML?**. Press **Cancel** to paste the content.

14. You will see that the UI is automatically populated on the right.

Swagger Editor

File Edit Generate Server Generate Client About

Try our new Editor

```
1 {
2   "swagger": "2.0",
3   "info": {
4     "version": "2.0",
5     "title": "Task Organizer",
6     "description": "Organize and maintain tasks"
7   },
8   "host": "5000.theiadocker-1-labs-prod-theiak8s-4-tor01.proxy.cognitiveclass.ai",
9   "paths": {
10    "/tasks": {
11      "get": {
12        "tags": [
13          "Tasks"
14        ],
15        "summary": "Returns a list of tasks.",
16        "description": "Optional extended description in Markdown.",
17        "produces": [
18          "application/json"
19        ],
20        "responses": {
21          "200": {
22            "description": "OK"
23          },
24          "405": {
25            "description": "Invalid Input"
26          }
27        }
28      },
29      "/task/{taskname}": {
30        "get": {
31          "tags": [
32            "Task specific activity"
33          ],
34          "summary": "Returns a task by name.",
35          "parameters": [
36
```

Task Organizer^{2.0}

[Base URL: lavanyas-5000.theiadocker-1-labs-prod-theiak8s-4-tor01.proxy.cognitiveclass.ai]

Organize and maintain tasks

Tasks

GET

/tasks Returns a list of tasks.

POST

/task Add a task

Task specific activity

GET

/task/{taskname} Returns a task by name.

DELETE

/task/{taskname} Delete a task by name.

15. Now you can test each of the endpoints. Four tasks have been already added for you, when the application was started. Click on the down arrow next to GET **tasks**.

GET

/tasks Returns a list of tasks.

16. Click on **Try it out**. This will allow you to try your REST API endpoint.

GET

/tasks Returns a list of tasks.

Optional extended description in Markdown.

Parameters

Try it out

No parameters

Responses

Response content type

application/json

Code	Description
200	OK
405	Invalid Input

17. Click on **Execute** to invoke a call to your REST API. This is a GET request which does not take any parameters. It returns the task as an **application/json**.

GET

/tasks Returns a list of tasks.

^

Optional extended description in Markdown.

Parameters

Cancel

No parameters

Execute

Responses

Response content type

application/json

^

Code	Description
200	OK
405	Invalid Input

18. You can scroll down to view the output of the API call.

Curl

```
curl -X 'GET' \
  'https://lavanyas-5000.theiadocker-0-labs-prod-theiak8s-4-tor01.proxy.cognitiveclass.ai/tasks'
-H 'accept: application/json'
```

Request URL

```
https://lavanyas-5000.theiadocker-0-labs-prod-theiak8s-4-tor01.proxy.cognitiveclass.ai/tasks
```

Server response

Code	Details
200	Response body <pre>{ "tasks": [{ "description": "Do the laundry this weekend", "name": "Laundry" }, { "description": "Finish assignment by Friday", "name": "Assignment" }, { "description": "Call family Sunday morning", "name": "Call family" }, { "description": "Pay the electricity and water bill", "name": "Pay bills" }] }</pre> Download

19. Try to do the following:

- Add a task
- Retrieve the tasks to see if your task is added to the list
- Get the details on one task
- Delete a task and check the list to verify that it is deleted.

20. From the **Edit** menu, click on **Clear Editor** to clear the content of the Swagger Editor.

Swagger Editor

File Edit Generate Server Generate Client About

1 {
2 "swagger": "2.0",
3 "info": {
4 "version": "1.0.0",
5 "title": "Task API",
6 "description": "API to manage tasks",
7 },
8 "host": "lavanyas-5000.theiadocker-0-labs-prod-theiak8s-4-tor01.proxy.cognitiveclass.ai",
9 "paths": {
10 "/tasks": {
11 "get": {
12 "tags": [

Import URL

Import file

Save as JSON

Convert and save as YAML

Clear editor

Task 2 - Build and Run Greetings API with Docker

In this task, you will set up a simple REST API server that responds with greetings in different languages, and for this we will use a pre-generated python-flask server package.

1. Open a new terminal and go to the `/home/project` directory.

```
cd /home/project
```

2. Download the Python-flask Server Code by executing the following command in the terminal.

```
wget -O python-flask-server-generated.zip "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/ml/jdlidmlhf5ytk5g0/python-flask-server-generated.zip"
```

3. Check to see if the zip file that you just downloaded, exists.

3 of 5

6/10/26, 5:05 PM

ls python-flask-server-generated.zip

4. Unzip the contents of the zip file into a directory named **python-flask-server-generated** by running the following command.

```
unzip python-flask-server-generated.zip -d python-flask-server-generated/
```

5. Change to the **python-flask-server** folder inside the folder you just extracted the zip file into.

```
cd python-flask-server-generated/python-flask-server-generated/python-flask-server-generated
```

6. The entire server setup along with endpoint is done for you already. Let's build the server code.

```
docker build -t mynewserver
```

This takes a while. If the build runs successfully you will have a new container with tag mynewserver.

7. Run the docker application now by running the following command. The server generated code automatically is configured to run on port 8080.

```
docker run -dp 8080:8080 mynewserver
```

You will get a hex code that indicates the application has started.

8. To confirm that the service is running and your REST API works, execute the following command.

```
curl localhost:8080/greetings
```

```
theia@theiadocker-lavanyas:/home/project/python-flask-server-generated/python-flask-server$ docker run -dp 8080:8080 mynewserver
7be538de67e2e81d1435c80cccd3e0c11fe03821e68517ed10e43f347ff89a37
theia@theiadocker-lavanyas:/home/project/python-flask-server-generated/python-flask-server$ curl localhost:8080/greetings
"do some magic!"
theia@theiadocker-lavanyas:/home/project/python-flask-server-generated/python-flask-server$
```

What you see in the output is what you have to do, **do some magic!**

• Click here for hint in case you encounter an error

- 1. Check Container Status

Open a terminal and run the following command to see if your container is running:

```
docker ps -a
```

- If the container is not running or has exited, note the STATUS and EXIT CODE.
- If it is running, confirm that the PORTS column shows:

```
0.0.0.0:8080->8080/tcp
```

This indicates that port 8080 inside the container is correctly mapped to port 8080 on your host machine.

- 2. Start the Container with Port Mapping

If the container was never started with port mapping, start it using the following command:

```
docker run --rm -p 8080:8080 mynewserver
```

This command ensures that your application inside the container is accessible from your local machine.

- 3. Once the container is running, test the `/greetings` endpoint using curl.

9. Now you should stop the server. For this you need the docker container id. Run the following command and copy the container id.

```
docker ps | grep mynewserver
```

```
$ docker ps | grep mynewserver
7be538de67e2 mynewserver "python3 -m swagger_" 44 minutes ago Up 44 minutes 0.0.0.0:8080->8080/tcp, :::8080
w->8080/tcp priceless_morse
```

10. To stop the container you need to kill the instance referring to the container id you copied in the last step.

```
docker kill <container_id>
```

11. In the file explorer go to, `python-flask-server-generated/python-flask-server/swagger_server/controllers/hello_in_different_languages_controller.py`. This is where you need to implement your actual response for the REST API.

